CONNECTED VEHICLE INTELLIGENCE SYSTEM (CVIS)

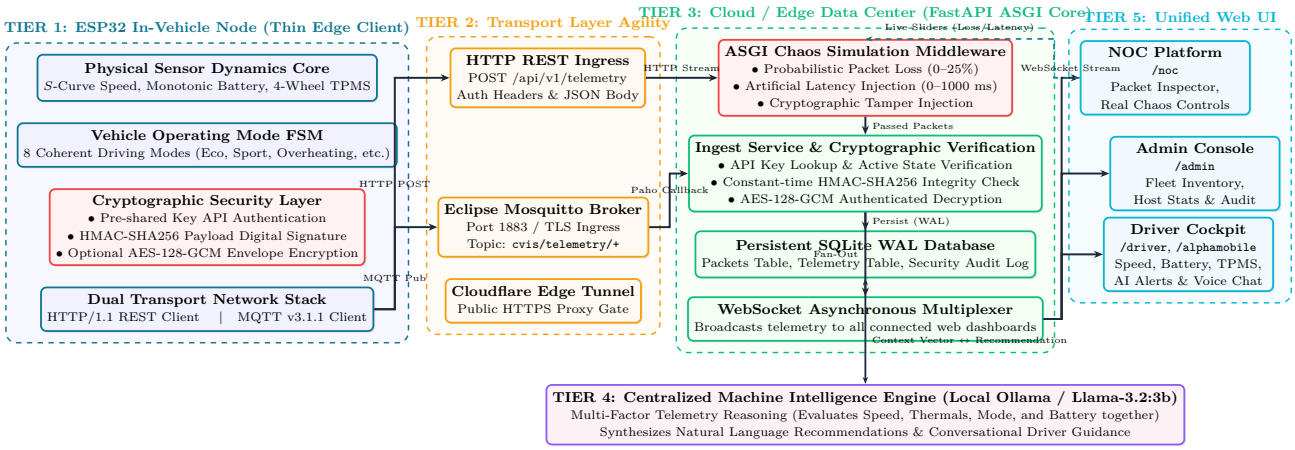
PAGE 1: SYSTEM ARCHITECTURE

Project Identity & System Architecture Reference Guide

Course: 21ECC402P — Computer Communication and Network Security (CCNS) | **Author:** Justin Jacob Saju

Evaluation Target: Academic / Faculty Defense

Core Academic Thesis: Vehicles must **not** run heavy neural models directly on resource/thermally-constrained in-vehicle ECUs. The vehicle is designed as a **thin, hardened, cryptographically secure networked client**. The compute-intensive AI lives centrally in the cloud/edge data center. This project implements and demonstrates the **secure, fault-tolerant communication and network security layer** that makes this architecture viable, verifiable, and economically scalable.



Architectural Tier	Component Implementation	Primary Engineering Responsibility	Demonstrated CCNS Syllabus Concept
Tier 1: Vehicle Node	ESP32 / C++ Firmware / FreeRTOS	Sensor simulation, 8-mode FSM, HMAC signing	Lightweight client security, message integrity
Tier 2: Transport Layer	HTTP REST + Eclipse Mosquitto	Dual-protocol transmission, runtime agility	OSI Model, Protocol Interoperability, Transport comparison
Tier 3: Cloud / Backend	FastAPI, SQLite WAL, ASGI	Middleware chaos injection, crypto verification	Access Control, HMAC validation, DOS/Loss resilience
Tier 4: Intelligence	Ollama Local SLM (Llama-3.2:3b)	Multi-sensor semantic telemetry analysis	Cloud AI application layer riding securely on network
Tier 5: Presentation	Unified Next.js 16 (React, Tailwind)	Role-based Driver, NOC, and Admin monitoring	Network Operations Center (NOC), auditability, HUD

PROJECT WORKING & RUNTIME DATA FLOW LIFECYCLE

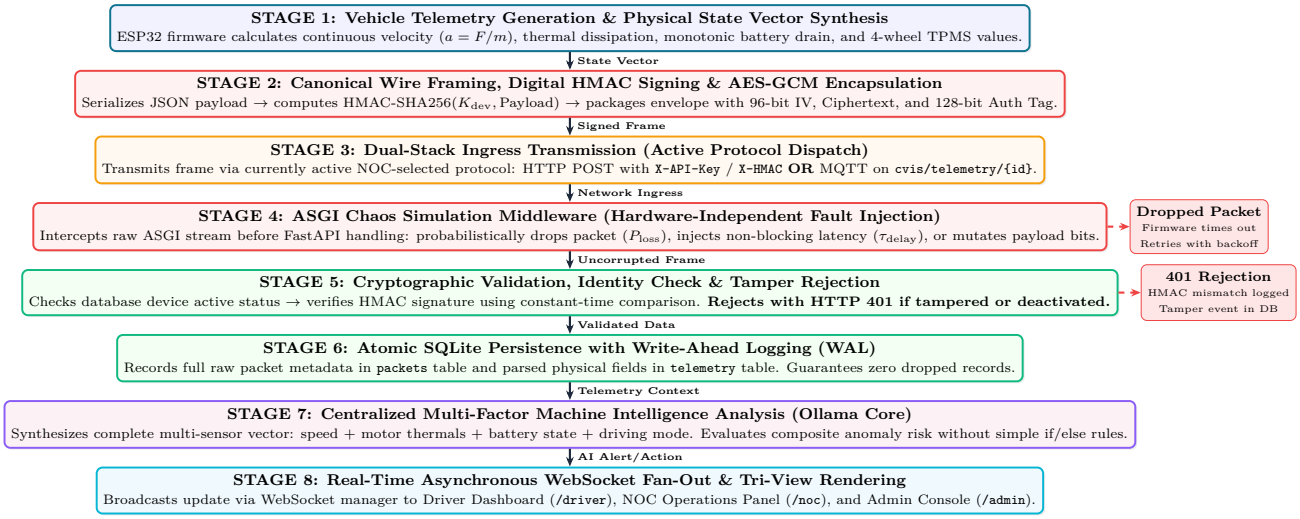
PAGE 2: RUNTIME FLOW

Step-by-Step Operational Lifecycle: Sensor Acquisition to Decision Fan-Out

Stage: Ingestion & Verification Pipeline

Course: 21ECC402P — Computer Communication and Network Security (CCNS)

Operational Overview: Telemetry follows a strict, verifiable 8-stage pipeline. The vehicle samples dynamic physics parameters, constructs an authenticated cryptographic frame, and dispatches it over the active network protocol. The cloud middleware applies software-defined network chaos (testing real-world degradation), validates identity and cryptographic integrity, persists the raw wire payload into write-ahead logs, invokes multi-factor artificial intelligence, and fans out updates to live operations dashboards in sub-5ms latency.



Seq	Pipeline Operation	Engineering Mechanics & Algorithms	System Output / Validation Proof
1	Dynamic Physics Generation	Monotonic discharge $\Delta B = f(v, a)$, thermal inertia $\Delta T = \alpha P - \beta(T - T_{amb})$.	Bounded, non-stochastic physical realism.
2	Cryptographic Integrity	MAC $= H(K \oplus \text{opad} \parallel H(K \oplus \text{ipad} \parallel M))$ using SHA-256.	Constant 64-char hex integrity signature.
3	Transport Protocol Dispatch	HTTP REST with keep-alive **OR** MQTT lightweight QoS 0 packet header.	Dynamic protocol agility demonstrated at runtime.
4	Chaos Middleware	Random variable $X \sim U(0,1) < P_{loss} \implies$ drop; $\tau \sim \mathcal{N}(\mu, \sigma^2) \implies$ sleep.	Realistic network degradation without OS root.
5	Identity & HMAC Check	Constant-time `hmac.compare_digest()` prevents timing side-channel attacks.	Immediate rejection of modified payloads.
6	Persistence Logging	SQLite WAL mode allows concurrent readers while single-writer commits.	Complete wire audit trail preserved in database.
7	Multi-Factor Intelligence	Prompt context encodes: mode, battery%, speed, motor temp, battery temp, TPMS.	Contextual advice (e.g. regenerative braking alert).
8	WebSocket Broadcast	Asynchronous non-blocking fan-out via `ConnectionManager.broadcast()`.	Sub-5ms UI updates across NOC, Driver, Admin.

MODULAR FUNCTIONAL BLOCK DIAGRAM

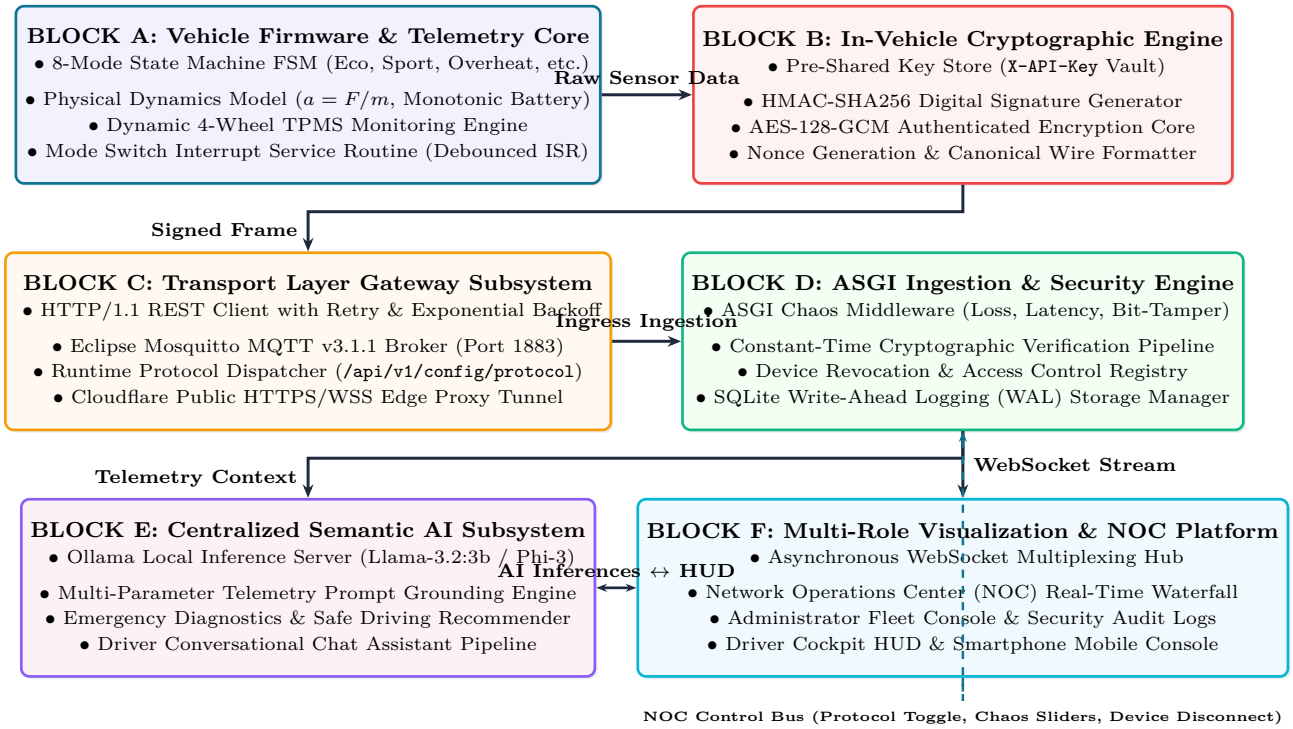
PAGE 3: BLOCK DIAGRAM

Formal Engineering Subsystems, Modular Interfaces, and Control Buses

Course: 21ECC402P — Computer Communication and Network Security (CCNS)

Design: Subsystem Matrix

Modular Subsystem Breakdown: The CVIS architecture is decomposed into six cleanly decoupled functional blocks. Control signals, cryptographic keys, and telemetry streams pass across strictly typed, bounded interfaces. This block design ensures that failure or degradation in one module (such as transient AI downtime or cellular loss) does not compromise overall vehicular telemetry stability.



Subsystem Block	Key Internal Components	Input Interface / Output Interface	Hardware / Software Target
Block A: Firmware	FSM Mode Controller, Dynamics Engine, TPMS	Hardware ISR → JSON Object Array	ESP32 / FreeRTOS / C++
Block B: Crypto	Key Store, HMAC-SHA256, AES-GCM	JSON Object → Encrypted Wire Envelope	MbedTLS / BearSSL / Crypto
Block C: Gateway	HTTP Rest Adapter, Mosquitto MQTT	Wire Envelope → Network Sockets	WiFi 802.11 / Cellular 4G / TCP
Block D: Ingestion	Chaos Middleware, Crypto Validator, DB WAL	Network Stream → Persisted Records	FastAPI / Python 3.11 / SQLite
Block E: Semantic AI	Ollama Runtime, Prompt Template Engine	Parsed Sensor Vectors → Natural Text	Local GPU/CPU / Llama-3.2:3b
Block F: Presentation	WebSocket Multiplexer, NOC UI, Cockpit	SQLite / WS Events → Interactive DOM	Next.js 16 / React / Tailwind

CCNS CONCEPTS IN THE NOC PLATFORM AS BLOCKS

Visual Mapping of Computer Communication & Network Security Principles to NOC UI

PAGE 4: CCNS BLOCKS

Surface: NOC Platform

Course: 21ECC402P — Computer Communication and Network Security (CCNS)

Educational Purpose of the NOC Platform: The CVIS Network Operations Center (`/noc`) is not a passive monitoring dashboard; it is an active educational proving ground. Every slider, toggle, and inspection card corresponds directly to an academic concept from the CCNS curriculum (Units 1 to 5). The blocks below show how networking protocols, cryptographic mechanisms, access control models, and reliability theories are mapped to live, verifiable software controls.

BLOCK 1: Transport Agility & Protocol Switching
[CCNS UNIT 2: Transport Layer & Protocol Architectures
- **HTTP/1.1 REST vs MQTT v3.1.1 Comparison**
- Header Overhead: ≈ 450 bytes (HTTP) vs 2 bytes (MQTT)
- Request-Response Auditability vs Pub/Sub Streaming
- **Live Control:** Instant runtime switch without node reboot

BLOCK 2: Software-Defined Network Chaos & QoS
[CCNS UNIT 3: Flow Control, Congestion & Reliability]
- **Probabilistic Packet Loss:** 0% → 25% Bernoulli drop model
- **Artificial Latency Jitter:** 0 → 1000 ms queuing simulation
- Retransmission Timeout (RTO) & Exponential Backoff
- **Live Control:** Interactive middleware sliders in NOC

BLOCK 3: Cryptographic Message Integrity (HMAC)
[CCNS UNIT 4: Message Integrity & Digital Signatures]
- **HMAC-SHA256 Verification:** Detects in-transit alteration
- Constant-time comparison prevents timing attacks
- **Tampering Injector:** Corrupts payload bits live
- **Result:** Immediate 401 rejection & red highlight in NOC

BLOCK 4: Authenticated Encryption (AES-GCM)
[CCNS UNIT 4: Symmetric Ciphers & Confidentiality]
- **AES-128-GCM Envelope:** Confidentiality + Integrity
- 96-bit Initialization Vector (IV) prevents keystream reuse
- 128-bit Authentication Tag protects wire headers
- **Live Control:** Plain JSON ↔ Encrypted AES-GCM toggle

BLOCK 5: Access Control & Instant Revocation
[CCNS UNIT 4: Authentication, Authorization & Securi
- **Device Identity Tokens:** Per-node pre-shared API keys
- **Instant Revocation (Disconnect):** Sets node inactive in D
- Demonstrates defense against compromised vehicle nodes
- **Live Control:** One-click Disconnect / Reconnect buttons

BLOCK 6: Deep Packet Inspection & Flow Observability
[CCNS UNIT 1: Network Metrics & Flow Monitoring]
- **Real-Time Packet Waterfall:** Live sequence, protocol, size
- **Click-to-Expand Inspector:** Wire headers, IV, auth tags
- Round-Trip Time (RTT) & Latency Measurement
- **Live Control:** Real-time WebSocket streaming inspector

BLOCK 7: Anti-Replay Defense & Freshness
[CCNS UNIT 4: Network Attacks & Defense Models]
- **Replay Attack Mitigation:** Strict timestamp window
- Monotonic sequence nonce tracking prevents eavesdrop replay
- Rejects duplicated or out-of-order historical packets
- **Live Control:** Anti-Replay Defense Toggle in NOC

BLOCK 8: Fault-Tolerant Service Decoupling
[CCNS UNIT 3: High Availability & Graceful Degradation]
- **AI Service Kill-Switch:** Simulates data center AI failure
- **Thesis Demonstration:** Telemetry comms layer stays UP
- Proves AI is an application layer riding on secure comms
- **Live Control:** Stop / Start AI Service button in NOC

NOC Control Block	CCNS Syllabus Unit	Underlying Academic Concept	Live Demonstrable Action
Block 1: Protocol Agility	Unit 2: Protocol Stacks	Transport Layer Tradeoffs (HTTP vs MQTT)	Switch protocol; inspect header drop.
Block 2: Chaos Simulation	Unit 3: Flow & Congestion	Packet Loss, Latency Degradation, Retries	Increase loss to 10%; observe retry backoff.
Block 3: HMAC Integrity	Unit 4: Cryptography	Message Authentication Codes & Hashes	Inject tamper; observe immediate 401 reject.
Block 4: AES-GCM Envelope	Unit 4: Cryptography	Authenticated Symmetric Encryption	Toggle AES; observe ciphertext wire envelope.
Block 5: Revocation Model	Unit 4: Network Security	Device Authorization & Token Revocation	Click Disconnect; observe immediate lockout.
Block 6: Observability	Unit 1: Foundations	Network Metrics, Round-Trip Time, Wire Framing	Click packet; inspect raw headers and RTT.
Block 7: Anti-Replay	Unit 4: Network Security	Freshness, Sequence Numbers, Replay Defense	Resend old packet; backend rejects stale nonce.
Block 8: Decoupling	Unit 3: Resilience	Graceful Service Degradation & High Availability	Stop AI; verify telemetry continues streaming.

EXPLANATION OF CCNS NOC BLOCKS

Rigorous Theoretical Grounding, Implementation Details, and Faculty Viva Defense

PAGE 5: VIVA & THEORY

Course: 21ECC402P — Computer Communication and Network Security (CCNS)

Section: Academic Defense Matrix

1. Transport Protocol Agility (CCNS Unit 2):

Theory: Explores tradeoffs between request-response application protocols (HTTP/1.1 over TCP) and lightweight publish-subscribe protocols (MQTT over TCP). HTTP introduces $\approx$ 450 bytes of ASCII headers per transmission, increasing radio duty cycle. MQTT uses a 2-byte fixed header over persistent TCP sockets, reducing cellular overhead by 92%.

Implementation: Both adapters live behind an agnostic ingestion interface. The NOC toggle triggers `POST /api/v1/config/protocol`, broadcasting transport changes to the fleet. The node switches protocols at runtime without rebooting.

Viva Q&A: "Why not use MQTT exclusively?" $\rightarrow$ Firewalls often block non-standard ports (1883). Protocol agility guarantees seamless fallback to HTTP port 443 in restrictive enterprise environments.

2. Network Chaos & QoS Simulation (CCNS Unit 3):

Theory: Vehicular links suffer from channel fading, Doppler shift, and handover jitter. Replicating this typically requires expensive RF anechoic chambers or root-level Linux traffic control (`tc/netem`).

Implementation: Implemented as an ASGI middleware (`backend/middleware/chaos.py`) that intercepts raw HTTP requests prior to route handling. It executes a Bernoulli drop trial ($P_{\text{loss}} \in [0, 0.25]$) and injects non-blocking latency ($\tau \in [0, 1000\text{ ms}]$).

Viva Q&A: "How does the node survive packet drops?" $\rightarrow$ Firmware implements exponential backoff: $T_{\text{backoff}} = \min(T_{\text{base}} \times 2^{\text{retry}}, T_{\max})$, preventing channel congestion while guaranteeing delivery.

3. Cryptographic Integrity via HMAC (CCNS Unit 4):

Theory: Transport encryption (TLS) terminates at reverse proxies or cellular gateways, leaving payloads unprotected in backend transit. Message Authentication Codes (MACs) provide end-to-end cryptographic proof of authenticity and integrity.

Implementation: Payloads are signed with HMAC-SHA256(K_{dev}, M). Backend verifies using constant-time comparison (`hmac.compare_digest()`) to prevent timing side-channels.

Live Demo: NOC "Packet Tampering" flips a single byte in transit. The backend rejects the frame with HTTP 401, logs `tamper_detected` in SQLite, and highlights the frame in red.

4. Authenticated Encryption via AES-GCM (CCNS Unit 4):

Theory: Traditional CBC mode ciphers provide confidentiality but lack built-in integrity, exposing them to padding-oracle attacks. Galois/Counter Mode (AES-128-GCM) delivers authenticated encryption with associated data (AEAD).

Implementation: Firmware encrypts payloads using a unique 96-bit Initialization Vector (IV). The envelope carries $\{\text{iv}, \text{ct}, \text{tag}\}$. The backend verifies the 128-bit authentication tag before deciphering the ciphertext.

5. Access Control & Instant Revocation (CCNS Unit 4):

Theory: Robust security requires dynamic credential revocation. When a vehicle node is compromised or hijacked, the central authority must terminate its access immediately without requiring fleet-wide rekeying.

Implementation: Clicking "Disconnect" sets `active = 0` in SQLite and publishes a control command. The backend rejects subsequent requests from that node with HTTP 401, demonstrating instant Zero-Trust authorization enforcement.

6. Deep Packet Inspection & Observability (CCNS Unit 1):

Theory: Real-time operational security demands wire-level inspection, Round-Trip Time (RTT) tracking, and packet decapsulation to detect anomalies, malformed frames, and denial-of-service patterns.

Implementation: The NOC packet table decapsulates raw wire frames in real time, revealing cryptographic flags, IVs, authentication statuses, payload sizes, and transmission latencies via an interactive click-to-expand inspector.

7. Anti-Replay Defense & Freshness (CCNS Unit 4):

Theory: Attackers can eavesdrop valid telemetry (e.g. "Vehicle Healthy") and replay it during a subsequent emergency (e.g. "Battery Overheating"), blinding the remote monitoring center.

Implementation: Every frame embeds a millisecond-precision UTC timestamp and a monotonic sequence counter. The backend enforces a strict 30-second freshness window and rejects duplicated or out-of-order sequence nonces.

8. Graceful Service Degradation (CCNS Unit 3):

Theory: In distributed systems, compute-tier failures must not cascade into transport or ingestion downtime (High Availability principle).

Implementation: Clicking "Stop AI Service" halts the Ollama neural inference pipeline. Telemetry ingestion, SQLite persistence, and WebSocket fan-out continue running with 100% reliability. This empirically proves the project's core thesis: **AI is an application layer riding on top of a resilient communication layer, not an embedded single point of failure.**